



Lista-7



a) Utilizando o modelo de advecção e difusão com assimilação de dados (kalman Filter) disponibilizado no github.

https://github.com/pkubota/pgmet_2025/tree/main/MET-348-3/LISTA_7/advec_diff_3dvar

```
real(real64), parameter :: sigma_b = 0.10_real64
```

```
real(real64), parameter :: sigma_obs = 0.02_real64
```

Discuta o comportamento do background (uc), solução verdadeira (ua) e análise (an) em função do erro da observação e o erro do modelo.



```
! kf_advec_diff.f90
program kf_advec_diff
  use Class_Fields , only: xa,ua
  !use Class_Fields_Adjoint, only: xa,ua
  USE ModAdvection , only : AnaliticFunction
  USE ModAdvection_Adjoint, only : AnaliticFunction_Adjoint
  USE Class_WritetoGrads, Only : SchemeWriteCtl,SchemeWriteData
  use Model , only: Init,Run_Model,Finalize
  use Model_Adjoint, only: Init_Adjoint,Run_Model_Adjoint,Finalize_Adjoint
  use LinearSolve, only :inverse
  use iso_fortran_env, only: real164
  implicit none
  integer , parameter :: nx = 60, nt = 400, nobs=nx
  real(real164), parameter :: L = 1.0_real164, u = 1.0_real164, kappa = 0.01_real164
  real(real164), parameter :: dx = L/(nx-1), dt = 0.25_real164*dx
  real(real164), parameter :: sigma_b = 0.10_real164
  real(real164), parameter :: corr_len = 0.15_real164
  real(real164), parameter :: sigma_obs = 0.02_real164
  real(real164), parameter :: q_diag = 1.0e-5_real164
  real(real164) :: Lcorr = 0.1_real164 ! comprimento de correlaçã
  LOGICAL :: monotone=.true.
```



b) Utilizando o modelo de advecção e difusão com assimilação de dados (4DVar) disponibilizado no github.

https://github.com/pkubota/pgmet_2025/tree/main/MET-348-3/LISTA_7/advect_diff_4dvar

```
real(real64), parameter :: sigma_b = 0.2_real64 ! desvio padrao do erro de background
```

```
real(real64), parameter :: sigma_obs = 0.02_real64 ! desvio padrao do erro de observacao
```

Discuta o comportamento do background (uc), solução verdadeira (ua) e análise (an) em função do erro da observação e o erro do modelo.



```
program advec_diff_4dvar_equiv
  use Class_WritetoGrads, only: SchemeWriteCtl, SchemeWriteData, InitClass_WritetoGrads
  !! 4D-Var forte para equacao 1D de adveccao-difusao
  !! Problema analogo a um KF 1D: mesmo tipo de verdade, observacoes, B, R e H.
  !!
  !!  $x_{n+1} = A x_n$  (A: operador de um passo do modelo)
  !!
  !!  $J(x_0) = 1/2 (x_0 - x_b)^T B^{-1} (x_0 - x_b)$ 
  !!  $+ 1/2 \sum_{k=1}^{win} [y_k - H A^k x_0]^T R^{-1} [y_k - H A^k x_0]$ 
  !!
  !! Solucao: (linear forte) resolvendo LHS  $x_0 = RHS$ , com
  !!  $LHS = B^{-1} + \sum_k (A^k)^T H^T R^{-1} H A^k$ 
  !!  $RHS = B^{-1} x_b + \sum_k (A^k)^T H^T R^{-1} y_k$ 
  !!
  use iso_fortran_env, only: real64
  implicit none

  ! -----
  ! parâmetros do problema
  ! -----

  integer, parameter :: nx = 60      ! pontos de grade
  integer, parameter :: nt = 100     ! tempo total (numero de passos)
  integer, parameter :: win = 10     ! janela de assimilacao (passos)
  integer, parameter :: nobs = 60    ! numero de pontos observados

  real(real64), parameter :: L = 1.0_real64 ! comprimento do dominio [0,L]
  real(real64), parameter :: c_adv = 1.0_real64 ! velocidade de adveccao
  real(real64), parameter :: k_dif = 0.01_real64 ! coeficiente de difusao
  real(real64), parameter :: dt = 0.001_real64 ! passo de tempo
  real(real64), parameter :: sigma_b = 0.2_real64 ! desvio padrao do erro de background
  real(real64), parameter :: Lcorr = 0.1_real64 ! comprimento de correlacao de B
  real(real64), parameter :: sigma_obs = 0.02_real64 ! desvio padrao do erro de observacao
  real(real64), parameter :: pi = 4.0_real64*atan(1.0_real64)
```